Advanced Rust 2026 - Lab 6: Systems Interfaces

Lukáš Hozda

Spring 2026

Setup

Create a library project:

```
cargo new --lib systems_lab
```

Add `libc`:

```
[dependencies]
libc = "0.2"
```

Use separate modules:

```
pub mod c_strings;
pub mod mapping;
pub mod pinned_record;
```

Exercise 1: C String Boundary

Implement checked helpers for crossing a C string boundary. The public API is safe. Internally, use `CString`, `CStr`, and `libc::strlen` where appropriate.

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum CStringError {
    InteriorNul(usize),
    MissingTerminator,
    InvalidUtf8,
}

pub fn to_c_bytes(input: &str) -> Result<Vec<u8>, CStringError> {
    todo!()
}

pub fn strlen_checked(bytes: &[u8]) -> Result<usize, CStringError> {
    todo!()
}

pub fn split_c_string_block(bytes: &[u8]) -> Result<Vec<String>, CStringError> {
    todo!()
}
```

Requirements:

1. `to_c_bytes` returns the UTF-8 bytes of `input` followed by one NUL byte.
2. `to_c_bytes` rejects interior NUL bytes and reports their byte index.

3. `strlen_checked` returns the number of bytes before the first NUL byte.
4. `strlen_checked` must reject an unterminated slice before calling C.
5. `split_c_string_block` parses consecutive NUL-terminated UTF-8 strings.
6. Empty strings inside the block are valid.
7. A non-empty block must end with NUL.
8. Invalid UTF-8 in a string returns `InvalidUtf8`.

Suggested checks:

```
assert_eq!(to_c_bytes("abc").unwrap(), b"abc\0");
assert_eq!(to_c_bytes("a\0b"), Err(CStringError::InteriorNul(1)));
assert_eq!(strlen_checked(b"abc\0ignored").unwrap(), 3);
assert_eq!(strlen_checked(b"abc"), Err(CStringError::MissingTerminator));
assert_eq!(split_c_string_block(b"red\0\0blue\0").unwrap(), vec!["red", "", "blue"]);
```

Exercise 2: Anonymous Memory Mapping

Implement an RAII wrapper around an anonymous writable memory mapping.

```
use std::io;
use std::ptr::NonNull;

pub struct Mapping {
    ptr: NonNull<u8>,
    len: usize,
}

impl Mapping {
    pub fn anonymous(len: usize) -> io::Result<Self> {
        todo!()
    }

    pub fn len(&self) -> usize {
        todo!()
    }

    pub fn as_slice(&self) -> &[u8] {
        todo!()
    }

    pub fn as_mut_slice(&mut self) -> &mut [u8] {
        todo!()
    }

    pub fn fill(&mut self, value: u8) {
        todo!()
    }
}

impl Drop for Mapping {
    fn drop(&mut self) {
        todo!()
    }
}
```

Requirements:

1. Use `libc::mmap` with `MAP_PRIVATE | MAP_ANONYMOUS`.
2. Use `PROT_READ | PROT_WRITE`.
3. Reject `len == 0` with `io::ErrorKind::InvalidInput`.
4. Treat `MAP_FAILED` as failure, not null.
5. Store the exact pointer and length returned by the successful mapping.
6. `Drop` must call `libc::munmap` with the same base pointer and length.
7. `as_slice` and `as_mut_slice` expose exactly `len` bytes.
8. Do not pass the pointer to Rust's allocator.

Suggested checks:

```
let mut mapping = Mapping::anonymous(16).unwrap();
assert_eq!(mapping.len(), 16);
mapping.fill(0xAB);
assert_eq!(mapping.as_slice(), &[0xAB; 16]);
mapping.as_mut_slice()[3] = 7;
assert_eq!(mapping.as_slice()[3], 7);
assert_eq!(Mapping::anonymous(0).unwrap_err().kind(), std::io::ErrorKind::InvalidInput);
```

Exercise 3: Pinned Self-Referential Record

Implement a small address-sensitive type. The object owns a `String` and stores a raw pointer to that field. Construction must pin the allocation before the internal pointer is initialized.

```
use std::marker::PhantomPinned;
use std::pin::Pin;

pub struct StableRecord {
    name: String,
    name_ptr: *const String,
    _pin: PhantomPinned,
}

impl StableRecord {
    pub fn new(name: String) -> Pin<Box<Self>> {
        todo!()
    }

    pub fn name(self: Pin<&Self>) -> &str {
        todo!()
    }

    pub fn pointer_matches(self: Pin<&Self>) -> bool {
        todo!()
    }

    pub fn rename(self: Pin<&mut Self>, name: String) {
        todo!()
    }
}
```

Requirements:

1. `StableRecord` must not implement `Unpin` automatically.

2. `new` returns `Pin<Box<StableRecord>>`.
3. Initialize `name_ptr` only after the value is pinned in the box.
4. `pointer_matches` checks whether `name_ptr` points to the current `name` field.
5. `rename` may replace the string value, but must keep the internal pointer valid.
6. Do not expose `&mut StableRecord` through the public API.

Suggested checks:

```
let mut record = StableRecord::new("alpha".to_string());
assert_eq!(record.as_ref().name(), "alpha");
assert!(record.as_ref().pointer_matches());
record.as_mut().rename("beta".to_string());
assert_eq!(record.as_ref().name(), "beta");
assert!(record.as_ref().pointer_matches());

let moved_pin = record;
assert_eq!(moved_pin.as_ref().name(), "beta");
assert!(moved_pin.as_ref().pointer_matches());
```